

从零训 Code LLM · Part 3(详细版)

大模型分布式训练与长上下文:一次完整推演,含所有往复

以 31B dense / 671B MoE / 1M-100M context 为标尺,从"单卡为何装不下"推到 TP/EP/FlashAttention/Ring/MLA。橙色块记录推演中的往复(错误假设 → 纠正 → 正确理解)——这些是推演最有价值的部分。

脉络(10步,每步含推演往复)

- 1. 显存拆解:单卡为何装不下
- 2. 张量并行 TP(往复:GQA 维度、内积降维、升降维通信)
- 3. 梯度检查点(往复:激活大头、存什么、重算粒度)
- 4. 数据并行 DP
- 5. 专家并行 EP(往复最多:TP/EP 混淆、传 token vs 专家、all-to-all vs ring)
- 6. FlashAttention(完整伪代码 + 多 head 展开 + 往复:类比 KV cache、是否近似、online softmax bug)
- 7. Ring Attention(完整伪代码 + seq 切 × head TP 两维叠加 + 往复:ring 传 vs all-reduce)
- 8. 100M:换算法
- 9. KV cache 规模经济
- 10. MLA(往复:同 GQA?PCA?attention 矩阵分解?)

1. 显存拆解:单卡为何装不下 31B

标尺:Gemma-4-31B——60层, $d_{\text{model}}=5376$,FFN 中间维 21504(GeGLU),32头($\text{head_dim}=256$),GQA KV 头 16,词表 262144,tie embedding。

一层参数

部分	矩阵	参数
Attention	$W_q\ 5376 \times 8192$	0.44亿
	$W_k\ 5376 \times 4096$	0.22亿
	$W_v\ 5376 \times 4096$	0.22亿
	$W_o\ 8192 \times 5376$	0.44亿
FFN	gate 5376×21504	1.16亿
	up 5376×21504	1.16亿
	down 21504×5376	1.16亿

一层 ≈ 4.79 亿(FFN 占 72%)。60层 $\approx 28.7\text{B} + \text{embedding}(\approx 14\text{亿}, \text{tie}) \approx 30.7\text{B}$ 。

训练显存

梯度 = loss 对每个参数的偏导,每参数一个 $\rightarrow$ 梯度量 = 参数量。AdamW 每参数存 m+v 两状态,FP32。

参数 BF16: 61G $\cdot$ 梯度 BF16: 61G $\cdot$ AdamW(m,v FP32): $30.7B \times 4 \times 2 = 246G(\text{大头}) \cdot$ 激活(见 § 3)

合计 $\sim 400G+ \rightarrow$ 单张 80G 装不下。

1.3 参数、梯度、优化器、激活的完整清单

以 30.7B 全参训练、BF16 混合精度为例,逐项:

- **参数(BF16)**:前向/反向用, $30.7B \times 2 = 61.4G$ 。
- **梯度(BF16)**:每参数一个偏导 $\partial L / \partial w$,梯度张量与参数张量**同形状**, $30.7B \times 2 = 61.4G$ 。
- **AdamW 一阶动量 m(FP32)**:梯度的指数移动平均(方向惯性), $30.7B \times 4 = 122.8G$ 。
- **AdamW 二阶动量 v(FP32)**:梯度平方的指数移动平均(自适应缩放), $30.7B \times 4 = 122.8G$ 。
- (可能的 FP32 master 参数: $30.7B \times 4 = 122.8G$,部分实现有)
- **激活(BF16)**:见 § 3,开 grad_ckpt 约 10-40G,不开 78G+。

固定开销(不含激活): $61.4 + 61.4 + 245.6 = 368.4G$ (+master 则 491G)。加激活 $\rightarrow \sim 400G+$ 。

为什么 m、v 要 FP32:它们是**累积统计**(指数移动平均),用 BF16 累加会因精度不足逐步丢失小量更新(大数吃小数)。FP32 保证累积精度。这也是全参训练显存大头($m+v = 245.6G$,比参数+梯度还大)。

推论:QLoRA 之所以省 —— base 冻结(不需梯度/优化器),只有小 LoRA($\sim$ 参数 0.7%)有梯度和 AdamW 状态,把 245.6G 的大头压到几 G。

1.4 推理 vs 训练的显存对比

	推理	训练(全参)
参数	61.4G	61.4G
梯度	—	61.4G
AdamW(m,v)	—	245.6G
激活	少量(单步)	78G+(或 grad_ckpt 10-40G)
KV cache	有($\propto$ context)	无(全序列一次算)
合计	$\sim 80G$ (可单卡)	$\sim 400G+$ (需多卡/QLoRA)

推演往复 ① 一问 AdamW 还是 Adam:确认是 AdamW(现代 LLM 标配)。AdamW 与 Adam 的区别仅在 weight decay 解耦(更新公式),**优化器状态一样大**(m,v 两份,246G),不影响显存。

2. 张量并行 TP:按 head 切

输入 X: (b, s, 5376)
 $Q=X \cdot W_q(5376, 8192)$ $K=X \cdot W_k(5376, 4096)$ $V=X \cdot W_v(5376, 4096)$

```
A=softmax(QKT/√d)  H=A·V  O=H·Wo(8192,5376)  X1=X+O
FFN: G=X1·Wg  U=X1·Wu(5376,21504)  M=gelu(G)⊙U  D=M·Wd(21504,5376)  X2=X1+D
```

推演往复 ②(GQA 维度) — 初判 W_k/W_v 第二维是 8192。纠正:GQA 下 Q 用 32 头(→8192),K/V 只用 16 头(→4096)。KV 头少一半是 GQA 省 KV cache 的点。

推演往复 ③(内积是否要降维) — 曾以为 "W_q 要先降维否则内积算不了"。纠正:内积在**每个 head 内部**进行,Q 每 head 256 维、K 每 head 256 维,head_dim 一致,直接内积。W_q 升到 8192 是因为 32 头,W_k 到 4096 是 16 头,但每个 head 都是 256,内积维度一致,**不需要降维**。

推演往复 ④(为什么 2 个 Q 共享 1 个 KV) — 不是"需要 2 个 Q",是 32 个 Q 头共享 16 个 KV 头 (32/16=2)。动机:KV cache ∝ KV 头数,KV 头少(16)→cache 减半省显存(长 context 尤甚);Q 头多 (32)保表达力。GQA = MHA(Q/K/V 都 32,cache 大)与 MQA(K/V 1 头,表达损)的折中。

切分与通信:升维本地算,降维 all-reduce

每卡持完整 X(相同副本),按 head 切权重:

- **升维**(W_q 5376→8192、W_g/W_u 5376→21504):每卡本地用完整 X × 自己那片权重,算自己那几个 head 的 Q(或 FFN 中间自己那片),**不通信**(权重训练前切好,升维输出天然各卡各占一部分维度)。
- **降维**(W_o 8192→5376、W_d 21504→5376):每卡从自己那片 × 对应行片 → **部分 O_i**;完整 O = ∑ 部分 O_i → **ring all-reduce 求和**。

推演往复 ⑤(升维到底本地还是集中,反复) — 曾在"5376→8192/21504 是每卡本地算,还是一张卡算完整"之间反复,措辞把"本地(不通信)"说成过"一张卡"。**最终厘清**:5376→8192 和 5376→21504 **处理完全一样**——都是升维,每卡本地算自己那片(2688 或 1024 维),不通信。区别只是那片大小,不是机制。**升维(5376→大)本地算不通信;降维(大→5376)才 all-reduce 求和**。

推演往复 ⑥(Wo 的 all-reduce 细节) — 每卡 H_i(b,s,1024) × W_o_i(1024,5376) = 部分 O_i(b,s,5376),8 份对应相加 = 完整 O。这一步用 **ring all-reduce**:环形连接的 8 卡分步累加,~8 步每卡都拿到完整和(不是汇聚到单卡再分发)。

推演往复 ⑦(每卡重复算升维 vs 传输) — 下一层 5376→8192 每卡本地各算自己那片(重复计算),而非"一张卡算完 copy 给其他卡"。前者省了 copy 传输,代价是重复计算;小投影时重复计算比跨卡 copy 便宜。

核心规律:升维本地算(不通信),降维 all-reduce(求和)。**完整 forward = 60 层外循环 × 每层(head TP:8 卡各算自己 head 的 attention + Wo all-reduce + FFN 切 + Wd all-reduce)**。TP 只在机内 (≤8 卡 NVLink,通信频繁),跨机不划算。

ring all-reduce 是软件算法(NCCL),跑在硬件链路(NVLink/NVSwitch)上——硬件提供高速全连接链路(A100/H100 SXM 的 NVSwitch 两两全连),软件把 ring 映射到物理拓扑。不是软件模拟硬件;物理链路本身也可组织成环形。

2.4 一步步看数据流(单层,head TP,8 卡)

以本卡(负责 4 个 head,head_dim=256)视角走一遍:

1. **输入**: X (b, s, 5376), 每卡都有完整副本(相同)。
2. **Q 投影(升维)**: 本卡持 W_q 的一片 = (5376, $4 \times 256 = 1024$)。算 $Q_{\text{local}} = X \cdot W_q_{\text{片}} \rightarrow (b, s, 1024)$ = 本卡 4 个 head 的 Q 。 **不通信**(本地算)。
3. **K/V 投影**: GQA 下本卡持 2 个 KV head $\rightarrow W_k/W_v$ 片 (5376, $2 \times 256 = 512$), $K_{\text{local}}/V_{\text{local}} = (b, s, 512)$ 。
4. **attention**: 本卡 4 个 Q head 配 2 个 KV head(组内 2:1, KV 广播对齐), 每 head 内 $QK^T \rightarrow \text{softmax} \rightarrow \times V$, 得 $H_{\text{local}} (b, s, 1024)$ 。每 head 独立。
5. **输出投影(降维)**: 本卡持 W_o 的行片 (1024, 5376)。算部分 $O_i = H_{\text{local}} \cdot W_o_{\text{片}} \rightarrow (b, s, 5376)$ 。**这是部分和**(只含本卡 4 head 的贡献)。
6. **all-reduce**: 8 卡的部分 O_i 求和 = 完整 $O (b, s, 5376)$ 。每卡都拿到完整 O 。
7. **残差**: $X_1 = X + O$ (每卡本地, X 和 O 都是完整的)。
8. **FFN**: 同理, W_g/W_u 列切(升维本地算 $G_{\text{local}}/U_{\text{local}} (b, s, 2688)$), W_d 行切(降维出部分 $D_i \rightarrow \text{all-reduce}$ 求和)。 $X_2 = X_1 + D$ 。

一层两次 all-reduce(W_o 后、 W_d 后)。60 层 $\rightarrow$ 120 次 all-reduce。这是 TP 通信频繁的来源,也是为什么 TP 必须机内 NVLink(每次 all-reduce 传 (b,s,5376) 量级数据,机内高带宽才扛得住;跨机网络会成瓶颈)。

参数 vs 激活 vs 输入长度(三者关系): 参数($W_q/W_k/W_v/\text{FFN}$)只由 d_{model} 定,与 **seq 长度无关**(所有 token 共享一套参数,复用)。seq 变长只增激活($s \times s$ 的 attention 矩阵、每 token 中间量),不增参数。60 层深能训靠**残差连接**(每层 $x+f(x)$, 梯度直通)。

3. 梯度检查点:激活大头是 s^2 的 attention 矩阵

推演往复 ⑧(激活大头是什么) — 初判"每层激活 = 层输出 (b,s,5376) $\approx$ 29MB, 60 层才 1.7GB, 不大"。

纠正: 漏了 **attention 分数矩阵 $A = QK^T$** , 形状 (b, heads, s, s), $\propto s^2$ 。s=2048、128 头:

$128 \times 2048 \times 2048 \times 2 \approx$ **1GB/层**。60 层不开检查点 $\approx$ **78GB** 激活 $\rightarrow$ 加参数就爆 80G。**激活大头是 s^2 的 A , 不是层输出**。

梯度检查点

标准: 60 层分若干段, 只在**段边界存输入**, 段内中间激活($Q/K/A/O/\text{FFN}$)前向算完即扔; 反向到某段从段边界**重新前向重建**。省显存, 代价 $\sim 25\%$ 慢(多一次前向)。

推演往复 ⑨(grad_ckpt 存什么) — 曾以为"存 $Q/K/O$ 供 BP 用"。**纠正**: 那是**不开** grad_ckpt 的做法(全存)。grad_ckpt 是**不存** $Q/K/A/O$, 只存段边界输入, BP 时重算。省显存靠"不存 + 重算"。

推演往复 ⑩(10 个点存什么) — 曾以为"10 个 checkpoint 点存 Q/K/A/O", 还算出 290MB。 **纠正**: 10 个点存的是 **段输入 X** (每点 ~29MB, 共 290MB), **不是 Q/K/A/O**; Q/K/A/O 在任何地方都不存, BP 重算。290MB 是常驻; BP 时有临时峰值 (重算段的 Q/K/A/O, 含 A 的 s^2 大头, 算完释放)。

推演往复 ⑪(发现标准做法的低效 → 细粒度改进) — 你指出: 标准"每 N 层一段", 反向段内越靠后的层要从段头重复 forward (BP 第 19 层从第 10 层段头算到 19, BP 第 18 层又从 10 算到 18), 段内 O (段长²) 重算。 **你的改进**: 每层都存输入 X (60 层 $\times$ 29MB $\approx$ 1.7GB, 不多) → BP 到某层直接从该层 X 只重算这一层的 Q/K/A/O, **每层只重算自己**, 重算最少。本质是更细粒度的 checkpoint (每层一个)。另一变体: 存 Q/K/O + 层 IO, **只不存 A (s^2 大头) 重算** — 本质接近 FlashAttention (不存完整 A)。

3.4 精确的激活显存估算

不开 grad_ckpt, 一层激活 (前向存着等反向) 粗算 ($b=8, s=4096, d=5376, \text{ffn}=21504$):

- Q/K/V/O 中间: $b \times s \times (\text{各投影维}) \times 2 \text{ 字节}$
- **attention 分数 A**: $b \times \text{heads} \times s \times s \times 2 = 8 \times 128 \times 2048 \times 2048 \times 2 \approx 1.07 \text{GB}$ (大头, $\propto s^2$)
- FFN 中间 (gate/up 的 21504 维): $b \times s \times 21504 \times 2 \times 2 \approx$ 几百 MB

一层 $\approx$ 1.3-1.5GB → 60 层 $\approx$ **78-90GB** → 单卡 80G 光激活就满, 加参数必爆。

理论 vs 实测差 1.7 倍: PyTorch caching allocator 预留 + 显存碎片 + 临时张量峰值 + 激活低估。DL 显存理论只估数量级, 精确值要实测。

权衡: 段长大 → 存的少但重算多 + 峰值大; 段长小 (每层) → 存的多 (1.7G) 但重算最少。X 不大时细粒度更优。BF16 是省显存关键 (激活 2 字节; 若 FP32 激活翻倍 78 → 156G)。

micro-batch 与段数的反比: 激活 $\propto$ batch $\times$ 段内激活。micro-batch=4 + 段=20, 等价于 micro-batch=1 + 段=5, 同样显存 (batch $\times$ 4 把段长缩到 1/4 抵消)。

4. 数据并行 DP

每卡 (组) 持模型副本, 处理不同数据 (batch 切分), 各算梯度, all-reduce 同步。 **梯度、AdamW 状态跟参数走** — 放同卡才能本地更新 ($W \leftarrow W - \text{lr} \cdot f(m, v)$), 不一致会额外跨卡搬运。

本推演的 MoE 设定里, **每卡本身持独立数据即已是 DP**; 每卡完整 X 就是这个 DP 单元的数据, 不需再单独分片 X。

推演往复 ⑫(DP 的定位) — 一度纠结"是否要再把 X 做 DP 分片"。厘清: MoE 每卡独立数据本身就是 DP, 不存在"再分 X"的问题。X 是否分片取决于 X 大小 (见 § 7: 1M 时 X 太大才 seq 切)。

5. 专家并行 EP: 每卡完整 X + 本地算 + 搬结果

671B MoE: 61 层, 每层 256 routed 专家 (+1 shared), top-8。专家是参数大头 (每层 MoE ~113 亿, 96%), dense 骨架 (attention+emb) 只 ~30B。

MoE 特性:每 token 只激活 8/256 专家 $\rightarrow$ 计算(前向+反向)都 $\propto$ 激活量(37B);反向只对激活过的专家算梯度(没选中的梯度为 0 不算)。但**显存不省**:全部 256 专家参数 + AdamW 要在显存待命 $\rightarrow$ 需 EP 分卡。

推演往复 ⑬(训练计算省不省) — 曾以为"MoE 只推理省,训练要对全部 671B 算导数"。**纠正:**反向只对**前向激活过的专家**算梯度(该 batch 没路由到的专家梯度为 0)。所以训练计算也 $\propto$ 激活(37B),**不只推理省**。但训练**显存**不省(全部专家待命)。

专家分卡

2048 卡 = 256 node $\times$ 8 卡。256 专家分 256 node $\rightarrow$ 每 node 1 专家/层。每 node 的 61 层专家 $\approx$ 26.9 亿参数,训练 $\approx$ 32G,node 内 8 卡分摊 $\approx$ 4G/卡。

推演往复 ⑭(专家要不要 TP 切 + 什么通信) — 曾提"每个专家用 8 卡 TP 按列切 + ring all-reduce"。**纠正:**混了 TP 和 EP。专家小(4404 万,单卡放得下),EP 是"一个专家整个在一处",不 TP 切;且 EP 的通信不是 all-reduce。

推演往复 ⑮(传专家 vs 传 token) — 曾提"ring 传 expert,只传 top-8 用到的"。**纠正:**① 一个 batch $2048 \times 8 = 16384$ 次选择撒到 256 专家,每专家平均被选 **64 次**,几乎所有 256 专家都被用到,"只传 top-8" $\approx$ 还是全部。② 传 1 个专家 88MB,而它对应的 token 平均 64 个 $\approx$ 0.9MB。**哪个小传哪个 $\rightarrow$ 传 token(小),不传专家(大),贵约 100 倍。**③ ring 传专家要遍历所有被用专家(串行多步),all-to-all 传 token 一步并行完成路由。

路由与通信(每卡完整 X 的方案)

每卡有完整 X。均摊:每专家平均被 **64 个 token** 选中($= s \times \text{top}_k / \text{专家数} = 2048 \times 8 / 256$)。

1. 每 node 的 e_i **本地**从完整 X 挑出"选中 e_i 的 ~ 64 个 token",算输出(其余 token 对 e_i 为 0)。因每卡有完整 X,**不用搬 token 进来**。
2. 每 node 产出"部分贡献"(b,s,7168,只有选了本专家的 token 非 0)。
3. **跨 node 求和**:每 token 的 D = 它 8 个专家(分散 8 node)贡献之和,256 份相加 = 完整 D。每卡得完整 $D_i, X_1 = X_2$ 。

推演往复 ⑯(每卡完整 X 是否成立 —— 判者反复用"标准 DP 分片"套,是判者的错) — 你早已设定"每卡独立/相同数据",但判者多次用"标准 MoE 是 DP 分片、每卡不同 token,要两次 all-to-all 搬 token"去质疑,拿一个不同前提否定你自洽的方案。**厘清并致歉后确认:**在你的设定(每卡完整 X)下,token 都在本地,只需一次"搬结果求和"(而非标准的搬进+搬回两次 all-to-all)。代价仅是每卡冗余存完整 X(但 X 小, $2048 \times 7168 \times 2 \approx 28\text{MB}$) + 每卡过一遍 router。**X 小时,这个局部性方案更优(省一次全局搬运)。**

5.4 均摊分析(为什么每专家 ~ 64 token)

精确均摊: $s = 2048$ token,每 token 选 top-8 $\rightarrow$ 总选择次数 = $2048 \times 8 = 16384$ 。均匀假设下每专家被选概率相等 $\rightarrow$ 每专家期望被选 = $16384 / 256 = 64$ 次。

公式:每专家期望 token 数 = (总 token $\times$ top_k) / 专家数 = $s \cdot k/E$ 。

每 node 的 e_i 非零输出 = $64 \text{ token} \times 7168 \times 2 \text{ 字节} \approx 0.9\text{MB}$;其余 $2048-64 = 1984$ 个 token 对 e_i 为 0。

负载均衡好时每专家 ~ 64 ;不平衡(有的专家被选多) $\rightarrow$ 卡负载不均 $\rightarrow$ 这就是 MoE 要 aux loss 做负载均衡的原因。

5.5 通信量估算

每 node 产出 64 个 {token, e_i 输出, 7168 向量} 的非零贡献 $\approx 0.9\text{MB}$ 。跨 node 求和:每 node 收发 $\sim 0.9\text{MB}$ 级,256 node 合计 $\approx 230\text{MB/层}$ (一个方向)。

对比"传专家"方案:传 256 个专家 $\times 88\text{MB} \approx 1.6\text{TB}$ —— 传 activation(230MB)比传 expert(1.6TB)小约 100 倍,这是"传小的"原则的量化体现。

求和用 ring:256 个 node 的部分贡献相加,用环形连接分步累加(类似 ring all-reduce 思路),而非全部汇聚到一个 node(那会成瓶颈 + 单 node 显存不够)。

结论:标准 EP 常与 DP 结合、每卡持不同 token $\rightarrow$ 两次 all-to-all(搬进算、搬回);每卡完整 X 方案 $\rightarrow$ 一次搬结果求和。哪个好看 X 大小:X 小(短 context) $\rightarrow$ 完整 X + 局部性更优;X 大(长 context)或大规模 DP $\rightarrow$ 分片。EP 的通信本质是 all-to-all(token 按目标专家重新分发),与 TP 的 all-reduce(求和)不同。

6. FlashAttention:从 O 的存储入手(完整推演)

6.1 问题:S 矩阵 $\propto s^2$,长序列存不下

标准 attention 三步:

$S = Q K^T / \sqrt{d}$	# (n, m) $\leftarrow$ 分数矩阵, $\propto s^2$
$P = \text{softmax}(S)$	# (n, m) 按行归一化
$O = P V$	# (n, d) 加权和

$n = \text{query 数}, m = \text{key 数}, d = \text{head_dim}$ 。S 是 (n,m),内存 $\propto s^2$ 。s=2048 单 head 才 8MB,但 128 头 $\times$ batch $\rightarrow$ GB 级;s=100 万时 $S = 10^{12}$ 元素,任何显存都存不下。这是长 context 的核心矛盾:计算 attention 必须有 QK^T ,但完整存 S 不可行。

6.2 突破口:不从 S/P 入手,从 O 入手

观察输出:O 只有 (n, d) —— 和序列长度 s 线性,不是平方。O 存得下。所以思路反过来:不去想"怎么存 S",而想"怎么在不存完整 S 的前提下算出 O"。

展开 softmax(第 i 行,减最大值防溢出):

$m_i = \max_j S_{ij}$ (第 i 行最大值)

$P_{ij} = \exp(S_{ij} - m_i) / \sum_k \exp(S_{ik} - m_i)$

$$O_i = \sum_j P_{ij} V_j = [\sum_j \exp(S_{ij} - m_i) V_j] / [\sum_k \exp(S_{ik} - m_i)]$$

关键:分子 $\sum_j \exp(...)V_j$ 和分母 $\sum_k \exp(...)$ 都是对 key(j/k)求和 → 能分块累加。

难点:① 分母要所有 key 的 exp 和(全局),分块时不完整;② 减的 m_i 是全局最大值,但算某块时后面的块还没看到,当前只能知道"到目前为止的最大值"(running max),后面来更大的要回头修正。这就是 online softmax 要解决的。

推演往复 ⑩(类比 KV cache?) — 曾觉得"从 O 累加、分块"这套"类似 KV cache"。**纠正:**方向相反。KV cache = **存**(缓存已算的 K/V 供后续复用,省重复计算);FlashAttention = **不存**(分块算 S,算完一块用于更新 O 就丢,省存储)。一个"存下来复用",一个"算完就扔",思路相反。

推演往复 ⑪(是否近似?) — 曾以为"FlashAttention 数学上是近似"。**纠正:精确。**online softmax 的 rescale 保证最终 O = 完整 softmax 的精确值(浮点重排可能有末位差异,数学等价)。**近似的是另一类:线性 attention**(Performer 用核函数、Linformer 用低秩投影,把 $\text{softmax}(QK^T)V$ 改写成 $\phi(Q)(\phi(K)^TV)$,先算 $\phi(K)^TV(d \times d)$ 避开 $s \times s$,计算降到 $O(s)$,但牺牲精度)。对比:**FlashAttention 精确** ($O(s^2)$ 计算 + $O(s)$ 存储,省的是存储不是计算);**线性 attention 近似** ($O(s)$ 计算+存储,省计算但有损)。

6.3 在线 softmax:running max / sum / O + rescale

把 K、V 沿 key 维分块(块 1..T)。对固定的 query 块 Q_i ,维护三个 running 量:**m(running max)**、**l(running sum,即分母)**、**O(running 输出,分子累加)**。每来一个 key 块 j:

```
初始: m = -inf, l = 0, O = 0(向量)
for j = 1..T:
    S_blk = Q_i · K_j^T / √d      # 遍历 key 块(Ring 里就是 ring 转来的每段 K/V)
    # 这一块的分值 (只这块,不存完整 S)
    m_new = max(m, rowmax(S_blk)) # 更新 running max
    p = exp(S_blk - m_new)         # 这块的未归一化权重(用新 max)
    scale = exp(m - m_new)         # 旧量的修正系数
    O = O * scale + p · V_j        # rescale 旧 O + 加新块
    l = l * scale + rowsum(p)      # rescale 旧 l + 加新块
    m = m_new
O_i = O / l                       # 最后除分母
```

rescale 的道理:running O 和 l 是"在旧 max m 下累加的"。新块带来更大的 max m_{new} ,之前所有项都少减了 $(m_{\text{new}} - m)$,即多乘了 $e^{(m_{\text{new}} - m)}$ 。**由于 O、l 是线性累加的,统一给它们乘 $e^{(m - m_{\text{new}})}$ 就等价于把之前每一项都重新按 m_{new} 归一**—— 不需要回头改已丢弃的旧块。这是 online softmax 的精髓:用一个标量 **rescale** 修正整个历史累加。

推演往复 ⑫(手写伪代码的 bug,逐个) — 初版伪代码有几处错:

- ① **w_t 忘了取 exp:**直接用原始分数 $S \times V$,应是 $\exp(S - m_{\text{new}}) \times V$ 。
- ② **O 覆盖没累加:**写成 $O_{it} = \dots$ (每个 t 覆盖),应是 $O += \dots$ (对 key 求和累加)。
- ③ **下标错:**把 O 写成按 key t 的 O_{it} ,实际 O 是按 query i 的(一个 query 一个 O 向量),key 维要 sum 掉。
- ④ **旧 O 没 rescale:**只 rescale 了 sum(l),漏了 running O 也要乘 scale。

⑤ **tmp_sum 用错 max**:写成 $\exp(\text{cur_max} - \text{new_max})$,应是 $\exp(S_t - \text{new_max})$ 。

纠正:大思路(分块 + running max/sum + rescale)对,但落地要精确 —— 上面 6.3 的伪代码是正确版。手生的是细节(exp、累加不覆盖、下标、旧 O 一起 rescale)。

6.4 多 head 的展开

上面伪代码是**单 head**的。实际每层多头(如 32 头),TP 下一卡持一部分头(如 16 头)。**每个 head 独立算自己的 O**:head 有各自的 Q/K/V(各自的投影),head 之间不共享、不通信。所以外面再套一层 head 循环:

```
for h = 1..16:                # 本卡负责的 16 个 head
  for i (query 块):          # 该 head 的 query 分块
    m, l, o = -inf, 0, 0
    for j (key 块):          # 该 head 的 key 分块 (Ring: ring 转来的段)
      ... online softmax(head h 的 Q_h, K_h, V_h) ...
      o_i^h = o / l
# 16 个 head 各得自己的 O^h,拼成本卡的输出片 (对应 8192 里的 16×512? 按 head_dim)
```

多 head 要点:① 每 head 独立算(各自 Q/K/V,各自 online softmax),**head 间无通信**;② 一卡 16 head 就是 16 次独立的 FlashAttention(可并行/批量,不是串行必须);③ 各 head 的 O^h 拼成本卡的输出片(本卡那部分 head 对应的维度);④ **head 之间的合并不在 attention 内部** —— 是所有卡的 head 输出 concat 后过 Wo(降维),那一步才有 TP 的 all-reduce(见 § 7 与 § 2)。

7. Ring Attention:1M context 的多卡 FlashAttention

7.1 为什么必须 seq 切

context = 1M 时,单卡先装不下 X: $X = 1M \times d(7168) \times 2\text{字节} \approx 13\text{GB}$ (还没算 K/V、激活、参数)。所以**按 seq(token 维)切**:8 卡各持 $1M/8 = 2^{17} \approx 13$ 万 token 的一段 X。

矛盾:attention 里,一段 Q(本卡的 13 万 query)要和**全部 1M token 的 K/V** 做内积(全局依赖,每个 query 要看所有 key)。但每卡只有自己那段的 K/V → **必须把其他 7 段的 K/V 拿过来**。

7.2 Ring 传 K/V = 内层 j 循环

把 § 6 的 FlashAttention 搬到多卡:

- **外层 i(各卡并行)**:8 卡各持一段 Q,各自独立跑 FlashAttention 的外层(每卡处理自己那 13 万 query 的 O)。
- **内层 j(ring 传 K/V)**:每卡把自己那段 K/V 沿环传给下一卡,转 8 次,每卡的 Q 就逐步见过全部 8 段 K/V。**每收到一段 K/V(对应 § 6 内层的一个 key 块),就跑一次 online softmax 更新本卡的 running m/l/O(rescale 旧 O + 加新块)**。8 次转完,每卡的 O_i 完整(见过所有 key)。

```
# Ring Attention,本卡视角(持 Q 段、初始 K/V 段)
m, l, o = -inf, 0, 0
cur_kv = 本卡的 K/V 段
for step = 1..8:                # ring 转 8 次
  S_blk = Q_local · cur_kv^T / √d  # 本卡 Q 和当前这段 K/V
```

```

m_new = max(m, rowmax(S_blk))
scale = exp(m - m_new)
O = O*scale + exp(S_blk-m_new)·cur_kv_V
l = l*scale + rowsum(exp(S_blk-m_new))
m = m_new
cur_kv = ring_send_recv(cur_kv)      # 把当前 K/V 传给下一卡,收上一卡的
O_local = O / l                      # 本卡这段 query 的输出

```

这就是 Ring Attention = FlashAttention 的多卡版:FlashAttention 的"分块遍历 key"变成"ring 转来的各段 K/V";online softmax 的在线累加机制原样照搬(rescale 旧 O + 加新块)。存储:每卡只存自己段的 X/O + 当前一段 K/V(线性),不存完整 S,也不存全部 K/V。

推演往复 ②(ring 传 vs all-reduce) — 曾说"ring-all-reduce 拿 K/V"。**纠正:**传 K/V 是 **ring 传(环形 send/rcv,点对点,每卡把 K/V 交给下一卡)**,不是 **all-reduce**(all-reduce 是求和归约)。这里没有求和 —— O_i 在**各卡本地**用 online softmax 累加,K/V 只是"绕环流动让每个 query 见到"。真正的 all-reduce 在别处:**Wo 投影之后**。各卡的 head 输出 concat 过 Wo(降维 8192→5376),Wo 按行切,每卡出部分 O,再 all-reduce 求和(§ 2 的 TP 规律)。所以:**attention 内部无 all-reduce(每 head 独立算);ring 传 K/V 是喂数据;all-reduce 在 Wo 之后(head TP 输出合并)**。

7.3 两个维度的并行叠加(seq 切 + head TP)

1M 训练同时用两维:

- **seq 切(Ring):**X 按 token 分到各卡(解决 X 装不下),ring 传 K/V。
- **head TP:**每卡内的注意力头再按 head 切(§ 2)。

所以完整一层:**60 层外循环 → 每层:各卡持自己的 seq 段 + 负责的 head → ring 传 K/V + 每 head 独立 online softmax 累加 O → head 输出 concat 过 Wo(all-reduce)→ FFN 切(all-reduce)**。

反向(BP):比前向繁 —— FlashAttention 反向要重新分块算 S(前向没存 S)来求梯度,Ring 反向也要再转一遍 K/V(或存中间统计)。但本质仍是**矩阵乘加 + 与前向对称的通信**(前向 ring 传/all-reduce 处,反向有对应操作),不改变大结构。理论上通信/计算模式和前向对称。

8. 100M:换算法,框架不变

1M/10M:Ring+Flash 撑得住(计算仍 $O(s^2)$ 但可切分,靠卡数/分块 tune)。**100M: $O(s^2)$ 爆炸**(标准 attention 光存 100M KV cache 要几百 H100/用户)→ **必须换算法**(线性/次二次序列算法,如 Magic LTM,比标准便宜约 1000 倍)。**但底层框架不变:**换的是 attention 算法,底下仍是矩阵乘 + reduce + 并行。实际 **1M + RAG 基本够**。

9. KV cache 规模经济

一层 $K+V = s \times d \times 2(K,V) \times 2$ 字节。1M、 $d=4096$ 、不压:一层 16GB,60 层 = **960GB**。GQA(K/V 减半到 2048):一层 8GB,60 层 = **480GB**。

推演往复 ②①(KV cache 算法) — 初写" $1M \times 4096 \times 2 = 8GB$ 。**厘清:**那是**单份(K 或 V)单层**。完整要 $\times 2(K,V \text{ 两份}) \times L(\text{层数})$ 。一层 $K+V=16GB$, 60 层 960GB(不压)。KV cache 每层都存(每层有自己 K/V)。

推演往复 ②②(规模服务显存 —— 按并发不按 DAU) — 曾按" $1M \text{ user} \times 100 \text{ session}$ "算,得天文数字。**厘清:**KV cache 占显存的是**同一时刻正在 decode 的并发请求数**,不是 DAU 或 session 总数(多数时刻 idle)。估算(1M DAU, 20 条/天, 每次 10 秒):平均并发 ≈ 2300 , 峰值 $\approx 7000-12000$ 。平均 50K context(非满 1M)、GQA:每请求 $\approx 6.5GB$, 峰值 1 万并发 $\approx 65TB \approx 800 \text{ 张 H100}$ 光存 cache。人人满 1M $\rightarrow 1.3PB$ 不可能。

结论:① 1M context 只给**付费/高净值用户**,普通用户短 context + RAG;② 平均 50K $\rightarrow 800$ 卡存 cache,加模型副本+计算+冗余(2-3 倍) ≈ 2400 卡,对 1M DAU 大厂可接受;③ 训推分离;④ 小模型可"模型+KV cache 同卡",大模型模型多卡(TP)、KV cache 跟着分布;⑤ KV cache 是长 context 大规模服务的核心瓶颈 $\rightarrow$ 需压缩(MLA)。

10. MLA:把 K/V 联合压成低秩 latent

推演往复 ②③(MLA 同 GQA 思路?) — 曾以为"MLA 和 GQA 同思路"。**纠正:**不同。GQA = **减少 KV 头数**(少存几个头,还是存原始 K/V);MLA = **把 K/V 压成低秩 latent**(存压缩表示,用时解压)。GQA 减头,MLA 压缩+重建。

推演往复 ②④(MLA 是 PCA?) — 曾以为"MLA 做主成分分析(PCA)"。**纠正:**不是。PCA 数据驱动(对数据算协方差取主成分,无学习);MLA 是**学出来的投影**—— W_{down} (压)/ W_{up} (解压)是训练参数,端到端学(类似 LoRA 的学低秩投影),不是 PCA。

推演往复 ②⑤(MLA 是 attention 矩阵分解?) — 曾归为"attention 矩阵的矩阵分解"。**纠正:**低秩压缩的**抽象大思路**相通,但 ① MLA 压的是 **K/V 向量**(每 token 的),不是 attention 矩阵 $S(QK^T)$;② MLA 是**学投影**,不是分解已有矩阵(PCA/SVD 才是分解)。**视角:MLA 是"压缩数据信息"**(学低秩表示保留 K/V 信息),不是存储技巧或矩阵分解。

机制与显存

把同一 token 的 K、V **联合压成一个共享低维 latent**(一个 latent 同时编码 K、V),只存这一份 latent。用时上投影解压: $K = \text{latent} \cdot W_{up_k}$, $V = \text{latent} \cdot W_{up_v}$ 。

显存(压 ~10 倍):latent 维 ≈ 400 ,只存一份联合 latent:一层 $= 1M \times 400 \times 2 = 0.8GB$,**60 层 $\approx 48GB$** (vs 不压 960GB、GQA 480GB,省约一个量级)。8 卡分摊 $\approx 6GB/\text{卡}$ 。

推演往复 ②⑥(先传还是先升维) — 1M 下 MLA 的 attention 计算仍是矩阵乘加 + reduce + Ring + Flash,多一步"从 latent 升维解压 K/V"。**顺序:**ring 传的是**压缩的 latent**(小,省通信带宽),传到本卡后**再升维解压**出 K/V,再算 attention。先传压缩的、本地升维,比先升维再传(传全维 K/V,带宽大)更优。

10.4 MLA 的数学(压 + 解压)

标准 MHA: 每 token 的 $x \rightarrow K = x \cdot W_k, V = x \cdot W_v$ (存 K、V 两个全维向量)。

MLA:

- **压(下投影):** $c = x \cdot W_{\text{down}}$ —— 把 x 压成一个低维 latent c (如维度 512, 远小于 K/V 的全维)。只存 c (一份, 联合编码 K、V 信息)。

- **解压(上投影):** 用时 $K = c \cdot W_{\text{up}_k}, V = c \cdot W_{\text{up}_v}$ —— 从同一个 c 分别升维出 K 和 V。
 $W_{\text{down}} / W_{\text{up}_k} / W_{\text{up}_v}$ 都是**训练学出来的参数**(端到端, 随 next-token 目标优化), 不是对数据做 PCA/SVD。类比 § Part2 的 LoRA (学低秩 B/A), MLA 也是学低秩投影。

省在哪: 标准存 K+V 两个全维 (如 2×4096); MLA 只存一个 latent c (如 512)。**KV cache 从 $2 \times d$ 降到 latent 维**, 压约 10-16 倍。且 W_{up} 可与后续矩阵吸收 (部分实现), 进一步省计算。

代价: 用时多一次上投影 ($c \rightarrow K/V$); 但换来 cache 大降 + 性能接近全 MHA (比 GQA 减头损失小)。

1M / 60 层显存: latent 512、只存一份 $\rightarrow$ 一层 = $1M \times 512 \times 2 \approx 1GB$, 60 层 $\approx 60GB$ (latent 400 则 $\sim 48GB$)。对比不压 960G、GQA 480G, MLA 省约一个量级。8 卡分摊 $\approx 6-8G/\text{卡}$ 。

Ring + MLA 的顺序(重要): 1M 下 seq 切 + ring 传时, **传的是压缩的 latent c (小, 省带宽), 传到本卡后再上投影解压出 K/V, 再算 attention**。先传压缩的、本地解压, 优于先解压再传全维 K/V (带宽大)。这把 MLA (省存储/带宽) 和 Ring (seq 切) 结合起来。

视角区别: GQA 存储视角 (减头); MLA 信息压缩视角 (学低秩表示, 保留 K/V 信息, 用时重建)。MLA 压得更狠 (接近全 MHA 性能, cache 小很多), 是长 context 推理省显存的关键。

小结

从 "单卡装不下" 出发: **TP** (head 切, 升维本地/降维 all-reduce) $\rightarrow$ **grad ckpt** (s^2 激活, 细粒度重算) $\rightarrow$ **DP** (独立数据) $\rightarrow$ **EP** (每卡完整 X + 本地算 + 搬结果) $\rightarrow$ **FlashAttention** (从 0 入手, online softmax, 精确) $\rightarrow$ **Ring Attention** (1M, seq 切 + ring 传 K/V) $\rightarrow$ **100M** (换算法) $\rightarrow$ **KV cache 规模经济** $\rightarrow$ **MLA** (K/V 联合压低秩 latent)。

贯穿框架: **矩阵乘 + reduce + 并行通信**。所有规模与优化都是在此框架上按显存/带宽约束做切分与压缩。而推演中的每一处往复 (GQA 维度、升降维通信、激活大头、grad_ckpt 存什么、EP 传 token vs 专家、Flash 是否近似、ring vs all-reduce、MLA vs PCA) —— 从错误假设到纠正, 才是真正理解的路径。

— Part 3 详细版完。含 26 处推演往复。 —